Recursive Language Models for Tool-Augmented Retrieval

Aryan Arora, Aadithya, Aarav, Aryan Jeena

Problem Statement/Motivation

LLMs struggle with long-context reasoning

As input length increases, attention diffuses across irrelevant tokens. Single-pass prompting fails on structured tasks like log parsing, multi-step retrieval, and aggregation

Technical Limitation

Irrelevant context wastes compute and increases hallucination risk, models lack iterative retrieval mechanisms

Core Challenge

Structured reasoning requires multi-step interaction with data, but standard LLM inference is a single forward pass with no external memory access.

Download:

Technical Approach

External Memory

The long context is stored outside the model in a Python environment. The model does not need to attend to every token at once.

Tool-Based Interaction

The agent generates Python code to query the context, the code runs in a sandboxed REPL, and the result is returned for the next step.

MDP Formulation

Tool use is modeled as a Markov Decision Process, the agent chooses either code or a final answer, and the reward balances correctness, steps, and token usage.

Download:

System Architecture (3 Components)

1

A task generator produces synthetic (context, question, answer) tuples.

2

The agent writes Python code to query the context.

3

A sandboxed REPL executes the code and returns output.

The long context lives in a Python variable, not in the prompt. The agent interacts with it through tool calls.

Download:

Task Types

We evaluate the system on three structured task families.

- Needle-in-Haystack requires finding a hidden piece of information
- KV Extraction requires filtering structured logs and extracting a key-value pair.
- Aggregation requires finding multiple metrics and summing their values.

All tasks are synthetically generated with known ground-truth answers.

Download:

Training Pipeline

Training occurs in two phases.

1. We perform Behavior Cloning using 500 expert trajectories generated by the heuristic agent.

We fine-tune Qwen2.5-Coder-1.5B with LoRA adapters for three epochs.

2. We apply REINFORCE.

The model generates code, the sandbox executes it, and reward is computed.

Policy gradients update the model toward higher-reward trajectories.

Download:

1. Task Distribution

Tasks sampled from:

$$(x, q, y^*) \sim \mathcal{D}.$$

Monte Carlo estimate:

$$E[f(X)] \approx \frac{1}{N} \sum_{i=1}^{N} f\left(x^{(i)}\right).$$

Download:

2. MDP Formulation

State:

$$s_t = (x, q, h_t).$$

Action:

$$a_t \in \{code, \{finalanswer\}.$$

Transition:

$$s_{t+1} = f(s_t, a_t).$$

Trajectory:

$$\tau = (s_1, a_1, \ldots, s_T, a_T).$$

Download:

3. Policy

$$\pi_\theta(a_t \mid s_t).$$

4. Reward (Lagrangian)

$$R(\tau) = C - \lambda_s \frac{T}{T_{\max}} - \lambda_t N_{tokens}.$$

Objective:

$$J(\theta) = E[R(\tau)].$$

Download:

5. Imitation Learning

Dataset:

$$\mathcal{D}_{BC} = \{(s_t, a_t)\}.$$

Loss (MLE):

$$\mathcal{L}_{BC} = -E[\log \pi_\theta(a \mid s)].$$

Download:

6. Policy Gradient (RL)

$$\nabla_\theta J(\theta) = E\left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,(R - b)\right]$$

7. Monte Carlo Training Loop

Sample trajectories $\tau^{(i)}$, compute $R^{(i)}$, update via:

$$\theta \leftarrow \theta + \eta\,\hat{\nabla}_\theta J.$$

Download:

Initial Results and Analysis

The trained local LLM (Qwen 1.5B, Behavior Cloning + RL) achieves:

- Needle-in-Haystack: 100% accuracy.
- KV Extraction: 90% accuracy.
- Aggregation: 50% accuracy.

The heuristic baseline achieves 100% across all tasks.

As expected, the LLM performs better in single to low step tasks.

Download:

Limitations

- Aggregation accuracy (50%) needs improvement
  - Model struggles with multi-step reasoning: find multiple keys → sum values → return result
- Small model (1.5B) limits code generation quality
  - Chosen for training speed and compute cost, but reduces capability vs larger models
- Only 3 synthetic task types, no real-world data
- Only trained for ~ 30 epochs

Download:

Next Steps

- Poker RLM setup
  - LLM + Python state (hand history, bets, stacks, board, opponent stats)
- What is trained
  - LLM policy: (state + context → action)
  - Actions: compute or output move (fold/call/raise)
- Data / long context
  - Multi-hand history
  - Full betting sequence
  - Table context (position, stacks)
- Reasoning flow
  - retrieve → compute (pot odds/equity) → act
- Training
  - Heuristic/solver → trajectories
  - Behavior cloning → imitation
  - RL (PPO) → optimize EV

Download:

Task: Poker

Download: